

Core Concepts — Why RAG, Chunking, Embeddings, Vector Search

Part 08 —  Retrieval-Augmented Generation · 08-rag/notes/01-core-concepts.md

1. Why RAG exists

An LLM's weights encode a snapshot of the internet up to some training cutoff, compressed and lossy. Three problems fall directly out of that fact, and RAG is the standard answer to all three:

Problem	Symptom	RAG's fix
Frozen knowledge	The model doesn't know about last week's incident, this quarter's pricing, or your product's v4 API	Retrieve the current document at query time; the model never needs to "know" it in advance
No provenance	The model states a fact with no way to check it	Retrieval gives you the source chunk — cite it, link it, let the user verify it
Private data	The model was never trained on your internal wiki, contracts, or support tickets (nor should it be)	Keep the data in your own index; the model only ever sees what you retrieve for a given query, under your access control

RAG vs. fine-tuning vs. long-context — pick on cost and freshness, not vibes

These three are not interchangeable, and the interview-question framing "RAG or fine-tuning?" is usually the wrong question. They solve different problems:

Approach	Teaches the model	Cost to update	Best for	Bad fit for
RAG	nothing — it stays frozen, you supply facts per-query	add/edit a document, done	fresh facts, citations, per-tenant private data, anything that changes	teaching a new <i>skill</i> , style, or output format
Fine-tuning	a skill, tone, or output format	retrain (hours–days, real cost)	consistent style/format, domain jargon, structured output habits	facts that change weekly — you'd be retraining constantly
Long-context (stuff everything in the prompt)	nothing, same as RAG	none — but pay per token, every call	small, static corpora that fit comfortably	large or growing corpora; see lost-in-the-middle in notes/02 [08-rag/notes/02-advanced-patterns-and-evaluation.md]

The opinion worth holding: RAG and fine-tuning are complementary, not competing — fine-tune the model to *use* retrieved context well and follow your citation format; use RAG to give it the facts. Reach for long-

context alone only when the corpus is small and static; the moment it grows or updates, you're paying full-context token cost on every call for information most of which is irrelevant to the current question, and you inherit lost-in-the-middle degradation for free. See Module 04 [04-llm] for why context windows behave this way mechanically.

The pipeline, end to end

```
documents → chunk → embed → store in vector index → [ingestion, offline]
query → embed query → retrieve top-k → (rerank) → assemble prompt → LLM → answer + citations
[online]
```

Ingestion happens once (and again on every update); retrieval happens on every single query. Most RAG quality problems are ingestion-time decisions that only become visible at serving time — which is exactly why chunking gets its own section below, first.

2. Chunking — where RAG quality is actually won or lost

You cannot embed and retrieve a whole 40-page document as one unit — it's too large, its embedding is a blurry average of everything in it, and the model can't afford to read all of it on every query. You split documents into **chunks**: the atomic unit of retrieval. Get this wrong and no downstream retriever, reranker, or bigger model fixes it, because the missing half of a fact was never retrieved in the first place.

`code/chunking_strategies.py` [08-rag/code/chunking_strategies.py] implements and compares four strategies on a real multi-paragraph document with one fact deliberately planted to straddle an unlucky chunk boundary:

Strategy	Mechanism	Failure mode it avoids	Failure mode it still has
Fixed-size	<code>text[i:i+size]</code> , nothing more	trivial to implement, fast	cuts words, sentences, and numbers in half with no awareness at all
Recursive	try the "nicest" separator (<code>\n\n</code> , then <code>.</code> , then <code>)</code> first, fall back only where needed	never splits inside a sentence unless the sentence itself exceeds chunk size	still a single arbitrary size for structurally different sections
Sentence-window	retrieval unit = one whole sentence; the LLM gets that sentence plus N neighbors	a sentence is <i>never</i> split, by construction — small unit for precise search, large unit for generation	more units to embed and index; window size is another parameter to tune
Structure-aware	split on document structure (Markdown headers, etc.), keep the header as metadata	a chunk never crosses a semantic section boundary; the header enables citation and section filtering	only as good as the document's actual structure — flat prose gets no benefit

The running script also sweeps chunk size on a labeled retrieval task and prints margin (the gap between the correct chunk's similarity and the best-scoring wrong chunk). Reading that curve: margin falls monotonically

as chunk size grows — a small chunk's vector isn't diluted by neighboring filler — but an arbitrarily small *fixed-size* chunk doesn't know where sentence boundaries are, so shrinking chunk size blindly only increases the odds of slicing tomorrow's fact in half. **The fix is boundary-respecting chunking at a moderate size, not an ever-smaller fixed-size chunk.**

Overlap

Recursive and fixed-size chunking usually carry a few dozen characters or tokens of **overlap** — the tail of chunk N is repeated at the start of chunk N+1 — so a reader who lands on chunk N+1 isn't missing the sentence that set it up. Overlap trades a little index bloat (duplicated text, more storage, slightly higher embedding cost) for fewer boundary casualties. A typical starting point is 10-20% of chunk size; tune it against your own fact-splitting rate, not a rule of thumb copied from a blog post.

Picking chunk size in practice

There is no universal right answer — it's a function of your embedding model's effective context, your documents' natural unit of thought, and your generation budget:

- **Too small** (a phrase, a single short sentence): high retrieval precision, but a retrieved chunk often lacks enough surrounding context for the LLM to answer confidently — it ends up hedging or guessing.
- **Too large** (a full section or document): diluted embeddings (the sweep above shows this directly), and every retrieved chunk burns far more of your context budget for the same signal.
- **The common production range** is 200-500 tokens per chunk with structure-aware or recursive splitting, and sentence-window or parent-document retrieval (see notes/02 [08-rag/notes/02-advanced-patterns-and-evaluation.md]) when you want the precision of a small unit and the context of a large one simultaneously.

3. Embeddings for retrieval — the short version (full treatment in Module 03)

An embedding model maps text to a dense vector such that semantically similar text lands close together. Module 03's notes [03-nlp/notes/01-text-to-vectors.md] cover static vs. contextual embeddings and how sentence embedding models are actually trained (contrastively — not by averaging token vectors); this module only adds the facts you need specifically for retrieval:

- **Normalize your vectors**, and cosine similarity becomes a plain dot product — cheaper to compute at index scale, and what every vector database assumes by default.
- **Asymmetric search needs asymmetric embeddings.** A query ("how do I reset my password") and a passage (a paragraph from a help doc) are different kinds of text. Modern retrieval models (E5, BGE, GTE, most commercial embedding APIs) are trained with **instruction prefixes** — literally prepend "query: " or "passage: " to the text before embedding. Get this backwards or omit it and retrieval quality degrades *silently* — nothing errors, your top-k is just quietly worse. This is one of the most common real-world RAG bugs, and it never shows up in a code review because the code runs fine.

- **Dimensionality is a cost/latency/quality knob**, not just a quality knob. Dimensions from 384 to 3072 are common; higher dimensions capture more nuance but cost more to store and compare at scale. **Matryoshka embeddings** (trained so that any length-prefix of the vector is still a valid, if lower-fidelity, embedding) let you truncate to a cheaper size at query time without re-embedding your whole corpus.
 - **The embedding model IS a retrieval hyperparameter**. Check the [MTEB leaderboard](#) before committing to one, and re-evaluate on *your* domain — general leaderboard rank does not guarantee it's the best fit for, say, legal contracts or source code.
-

4. Vector search mechanics

Once documents are chunked and embedded, you need to find, for a query vector, the k nearest document vectors. Two fundamentally different approaches:

Exact search (kNN / brute force)

Compare the query vector against every single stored vector, rank by similarity, take the top- k . Perfect recall by definition — there's no approximation to be wrong about. The cost is $O(n \cdot d)$ per query (n vectors, d dimensions), which is fine up to roughly tens of thousands of vectors on a single machine and increasingly impractical beyond a few million. Some vector databases (pgvector without an index, small FAISS `IndexFlatL2`) run this mode by default — know when you're doing it.

Approximate Nearest Neighbor (ANN) search

At real scale you trade a small amount of recall for a large amount of speed. Two index families dominate:

HNSW (Hierarchical Navigable Small World) — builds a multi-layer graph where each vector is a node connected to its approximate nearest neighbors; search starts at a sparse top layer and greedily descends, narrowing in on the right neighborhood. This is the default in most modern vector databases (Qdrant, Weaviate, pgvector's `hnsw` index type, Chroma) because it gives excellent recall at low latency without needing to know your data's distribution ahead of time. Cost: builds a large in-memory graph structure, and insertion is slower than a flat index — HNSW is read-optimized.

IVF (Inverted File Index) — clusters vectors into `nlist` partitions (via k-means) at index-build time; a query only searches the `nprobe` partitions nearest the query vector, not all of them. Cheaper to build and more memory-efficient than HNSW, but recall is more sensitive to how well the clusters match your actual query distribution, and it needs a representative sample of your data upfront to build good clusters (a cold-start problem HNSW doesn't have). Often paired with product quantization (`IVF-PQ`) to compress vectors and cut memory further, at a further small recall cost.

The triangle you're actually trading against



Every ANN knob (HNSW's `ef_search/M`, IVF's `nlist/nprobe`) moves you along this triangle. Concretely:

Lever	Turn it up...	...and you get	...at the cost of
HNSW <code>ef_search</code>	higher	better recall	higher query latency
HNSW <code>M</code> (graph connectivity)	higher	better recall, faster search	much more memory at build time
IVF <code>nprobe</code>	higher	better recall	higher query latency (searches more partitions)
IVF <code>nlist</code>	higher (more, smaller clusters)	can improve recall per probe	more clusters to compute distances to at index-build and query time

The practical workflow: don't guess these values. Build a small labeled retrieval eval set (see `code/rag_eval_metrics.py` [08-rag/code/rag_eval_metrics.py] and `notes/02` [08-rag/notes/02-advanced-patterns-and-evaluation.md] §6), sweep the index's recall knob, and pick the cheapest setting that clears your `Recall@k / NDCG@k` bar. A 99% recall ANN index that's 20x faster than exact search is very often the right trade — but only if you measured the 99%, not assumed it.

Metadata filtering

Real queries are rarely "find the *k* nearest vectors" alone — they're "find the *k* nearest vectors *where tenant_id = X and published_after = 2025-01-01*". This is **metadata filtering**, and how it's applied matters a lot for both correctness and cost:

- **Post-filtering** (retrieve top-*k*, then discard results that fail the filter) is simple but can return fewer than *k* results — or zero — if the filter is selective and the true matches don't happen to be in the initial unfiltered top-*k*.
- **Pre-filtering** (restrict the ANN search to only the matching subset before ranking) gives correct counts but can break the ANN index's assumptions — some indexes only support this efficiently for specific filter types, or fall back to exact search within the filtered subset.
- Production vector databases increasingly offer **filtered HNSW** variants that integrate the filter into graph traversal itself, avoiding both failure modes — check whether the one you're evaluating actually does this, or is silently post-filtering and calling it "metadata support."

For multi-tenant systems, metadata filtering on `tenant_id` is not optional — it's the access-control boundary. An index without a *correct* filtering strategy is a data leak waiting for the wrong query.

5. Where this returns

Idea here	Where it returns
Embeddings, cosine similarity	Module 03 — NLP [03-nlp/notes/01-text-to-vectors.md] (full training/theory treatment)
Context budget spent on retrieved chunks	Module 04 — LLM [04-llm] (context windows, lost-in-the-middle)
Chunking + retrieval as an agent tool	Module 06 — AI Agents [06-ai-agents] (agentic/iterative RAG)
Hybrid search, rerank, advanced patterns, GraphRAG, eval, failure modes	notes/02-advanced-patterns-and-evaluation.md [08-rag/notes/02-advanced-patterns-and-evaluation.md]
Recall@k / MRR / NDCG@k, faithfulness	Module 15 — AI Evals [15-ai-evals] (production-grade eval methodology)